Russo Paola N39001631 - Vitale Luciano M63001774

Corso di Algoritmi e Strutture Dati – Homework set #1

ES. 1.1 Notazione Asintotica

- *f(n) = O($f(n)^2$)*
  SEMPRE VERA
  Dato n numero naturale, $f(n)^2$ sarà sempre un limite superiore asintotico per $f(n)$ in quanto $f(n)^2$ è un infinito di ordine superiore rispetto a $f(n)$ per funzioni asintoticamente non negative e crescenti.
  In particolare, risulta sempre vero: $0 \leq f(n) \leq cf(n)^2 \ \forall n \geq n_0$ con n_0, $c>0$.

- *f(n) + O(f(n)) = Θ(f(n))*
  SEMPRE FALSA
  O(f(n)) è l'insieme di tutte le funzioni dominate da f(n). Poiché il limite è stretto abbiamo che f(n) + Of((n)) = O(f(n))
  La relazione O(f(n)) = $\Theta f(n)$ è falsa perché Θ(f(n)) ⊆ O(f(n)) ma non vale il viceversa.

- *f(n) = Ω(g(n)) e f(n) = o(g(n))*
  SEMPRE FALSA
  la definizione di Ω(g(n)), g(n) è un limite inferiore asintotico per f(n), mentre per o(g(n)), g(n) è un limite superiore ma non è asintoticamente stretto. Per cui le relazioni:
  o: f(n) < cg(n)
  Ω: cg(n) ≤ f(n)
  risultano incompatibili.

ES. 1.2 Complessità

- *È vero che* 2^{n+1} *= O(2^n)?* SI'

Asintoticamente $2^{n+1} = 2^n$ quindi possiamo riscrivere l'equazione come: f(n) = O(f(n)), che è una relazione vera per la proprietà riflessiva

- *È vero che* $2^{2n} = O(2^n)$*?* NO

Ponendo 2^n= f(n) otteniamo $2^{2n} = (2^n)^2 = f(n)^2$ quindi risulterebbe $f(n)^2 = O(f(n))$ che è chiaramente falsa.

ES. 1.3 Ricorrenze

- *T(n) = 2T(n/3) + n log(n)*

Utilizziamo il Teorema dell'Esperto, in particolare il 3° caso.

$n^{\log_b a} = n^{\log_3 2} = n^{0.63}$

$f(n) = n \log n = \Omega\left(n^{\log_3 2+\varepsilon}\right) = \Omega(n)$ con $\varepsilon = 0.4$

In particolare, quest'ultima relazione ci assicura che f(n) è polinomialmente più grande di $n^{\log_b a}$, ovvero riusciamo a trovare un fattore n^{ϵ} che fa valere la relazione di sopra.

Inoltre, deve valere: $a[f(n/b)] \leq c(f(n))$ per $c < 1$ e n sufficientemente grande:

$2(n/3) \log(n/3) \leq c\, n \log n$ con c=2/3 otteniamo

$2(n/3) \log(n/3) \leq (2/3)\, n \log(n)$

Semplificando otteniamo $\log(n/3) \leq \log(n)$ che è sicuramente vero in quanto la funzione log n è crescente => $T(n) = \Theta(n \log n)$

- $T(n) = 3T(n/5) + \log^2 n$

$$n^{\log_b a} = n^{\log_5 3} = n^{0.68}$$

$f(n) = \log^2 n$

Non rientra in nessuno dei casi del Teorema dell'Esperto in quanto non riusciamo mai a trovare un fattore n^{ϵ} che renda f(n) polinomialmente più piccola o più grande di $n^{\log_b a}$

Utilizziamo il metodo dell'albero di ricorrenze.

$T(n) = 3T(n/5) + \log^2(n)$

Effettuo cambio di variabile ponendo:

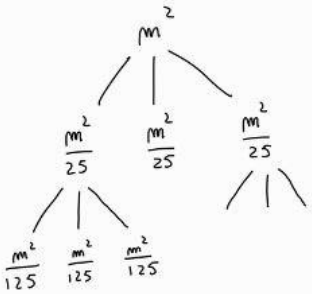
$m = \log(n) \Rightarrow 2^m = n$, ottenendo:

$T(2^m) = 3T\left(\frac{2^m}{5}\right) + m^2$

Pongo $T(2^m) = S(m)$ ed ottengo la nuova ricorrenza:

$S(m) = 3T\left(\frac{m}{5}\right) + m^2$

Costruisco l'albero:



Poiché assumiamo un costo costante per $n \leq 10$

Dobbiamo fermarci quando $T\left(\frac{m}{5^i}\right) = T(10)$

Quindi al livello $\frac{m}{5^i} = 10 \Rightarrow i = \log_5\left(\frac{m}{10}\right)$

Il costo dell'albero è il costo del caso base più il costo su ogni livello:

$$\text{Caso base} = 3^i T(c)\Big|_{i = \log_5\left(\frac{m}{10}\right)} = \Theta\left(3^{\log_5\left(\frac{m}{10}\right)}\right) = \Theta\left(3^{\log_5 m - \log_5 10}\right) = \Theta\left(\frac{3^{\log_5 m}}{3^{\log_5 10}}\right)$$

$$= \Theta\left(3^{\log_5 m}\right)$$

$$\text{Costo livelli} = \sum_{i=0}^{\log_5\left(\frac{m}{10}\right)-1} 3^i \frac{m^2}{5^i}$$

$$\Rightarrow S(m) = \sum_{i=0}^{\log_5\left(\frac{m}{10}\right)-1} 3^i \frac{m^2}{5^i} + \Theta\left(3^{\log_5 m}\right) \leq \sum_{i=0}^{\infty} 3^i \frac{m^2}{5^i} + \Theta\left(3^{\log_5 m}\right) \leq$$

$$\leq \sum_{i=0}^{\infty} \left(\frac{3}{5}\right)^i m^2 + \Theta\left(3^{\log_5 m}\right) = \frac{1}{1-\frac{3}{5}} m^2 + \Theta\left(3^{\log_5 m}\right) =$$

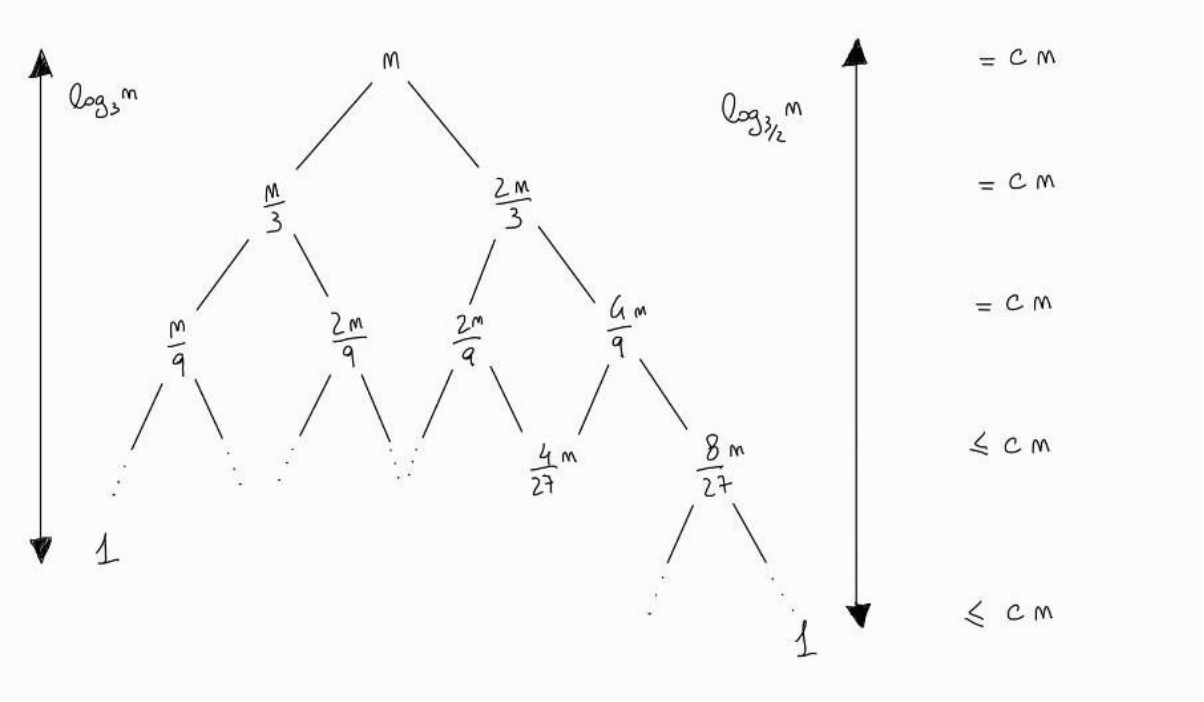
$$= \frac{5}{2} m^2 + \Theta\left(m^{\log_5 3}\right) \Rightarrow S(m) = O(m^2)$$

$$\Rightarrow T(2^m) = O(\log^2(n))$$

ES. 1.5 Ricorrenze

- *Utilizzando l'albero di ricorsione, dimostrate che la soluzione della ricorrenza* $T(n) = T(n/3) + T(2n/3) + cn$*, dove c è una costante, è* $\Omega(n \log n)$.

Poiché in questa ricorrenza i rapporti di divisione del sottoproblema sono diversi, segue che il caso base verrà raggiunto in tempi diversi nell'albero all'estrema sinistra e nell'albero all'estrema destra. Questo di traduce in un differente numero di livelli dell'albero andando da sinistra a destra. In particolare, quello di sinistra avrà una profondità $log_3(n)$, quello di destra $log_{3/2}(n)$.



Di conseguenza per raggiungere il caso base verrà impiegato in qualsiasi caso un tempo proporzionale al numero di livelli percorsi che abbiamo visto essere sempre un logaritmo. Poiché qualsiasi logaritmo cresce alla velocità di log(n), possiamo generalizzare dicendo che in questo albero abbiamo $\Theta(\log \quad n)$ livelli. Ogni livello ha un costo di al più cn, quindi

Da SX abbiamo: $T(n) \geq (\ log_3(n)+1)\ cn = \Theta(n \log n) \Rightarrow T(n)=\Omega(n \log n)$

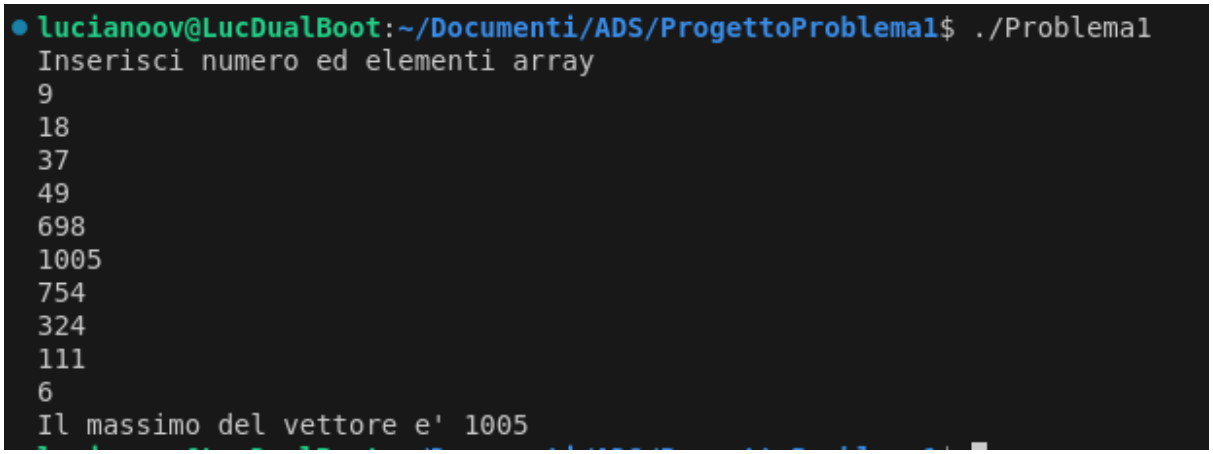
Da DX abbiamo: $T(n) \leq (\ log_{\frac{3}{2}}(n) + 1)\ cn = \Theta(n \log n) \Rightarrow T(n)=O(n \log n)$

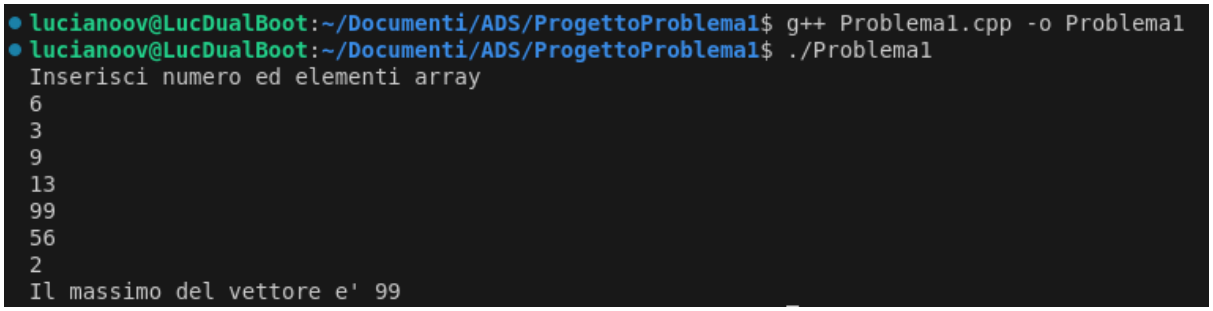
Abbiamo dimostrato che $T(n)=\Theta(n \log n)$ => è anche $T(n)=\Omega(n \log n)$.

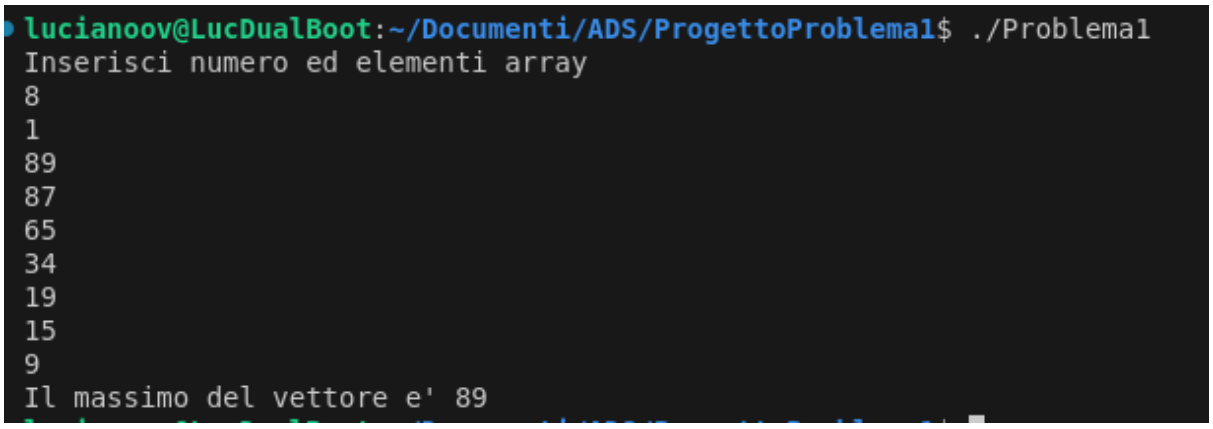
Problemi

Si riportano di seguito esclusivamente i casi di test dei 3 problemi. Il codice e l'indicazione della complessità è riportata nei file .txt allegati.

1. Ricerca del massimo in un array unimodale







2. Ricerca del prefisso comune più lungo date n stringhe

```
lucianoov@LucDualBoot:~/Documenti/ADS/ProgettoProblema2$ ./problema2
Inserisci numero e stringhe da analizzare
3
lancia
tavolo
termosifone

Il prefisso in comune piu' lungo e'
```

```
lucianoov@LucDualBoot:~/Documenti/ADS/ProgettoProblema2$ ./problema2
Inserisci numero e stringhe da analizzare
5
portogallo
porto
portobello
portofino
portone

Il prefisso in comune piu' lungo e' porto
```

```
lucianoov@LucDualBoot:~/Documenti/ADS/ProgettoProblema2$ g++ soluzione.cpp -o problema2
lucianoov@LucDualBoot:~/Documenti/ADS/ProgettoProblema2$ ./problema2
Inserisci numero e stringhe da analizzare
6
giocattolo
giocoliere
giocondo
gioconda
gioco
giocare

Il prefisso in comune piu' lungo e' gioc
```

3. Implementazione della proceduta Treap_Insert

```
lucianoov@LucDualBoot:~/Documenti/ADS/ProgettoProblema3$ g++ Treap.cpp -o Treap
lucianoov@LucDualBoot:~/Documenti/ADS/ProgettoProblema3$ ./Treap
Quanti elementi vuoi inserire nel Treap? 10
Inserisci le chiavi
Carattere 1: g 4
Carattere 2: b 7
Carattere 3: a 10
Carattere 4: e 23
Carattere 5: c 25
Carattere 6: h 5
Carattere 7: k 65
Carattere 8: i 73
Carattere 9: d 9
Carattere 10: f 2
└─f:2
    ├─g:4
    │   ├─h:5
    │   │   ├─k:65
    │   │   │   ├─-
    │   │   │   └─i:73
    │   │   │       ├─-
    │   │   │       └─-
    │   │   └─-
    │   └─-
    └─b:7
        ├─d:9
        │   ├─e:23
        │   │   ├─-
        │   │   └─-
        │   └─c:25
        │       ├─-
        │       └─-
        └─a:10
            ├─-
            └─-
```

```
lucianoov@LucDualBoot:~/Documenti/ADS/ProgettoProblema3$ ./Treap
Quanti elementi vuoi inserire nel Treap? 7
Inserisci le chiavi
Carattere 1: p
Carattere 2: a
Carattere 3: o
Carattere 4: l
Carattere 5: i
Carattere 6: t
Carattere 7: e
└─e:19
    ├─o:32
    │   ├─p:40
    │   │   ├─t:45
    │   │   │   ├─-
    │   │   │   └─-
    │   │   └─-
    │   └─i:66
    │       ├─l:87
    │       │   ├─-
    │       │   └─-
    │       └─-
    └─a:58
        ├─-
        └─-
```


lucianoov@LucDualBoot:~/Documenti/ADS/ProgettoProblema3$ ./Treap
Quanti elementi vuoi inserire nel Treap? 7
Inserisci le chiavi
Carattere 1: l
Carattere 2: u
Carattere 3: c
Carattere 4: i
Carattere 5: a
Carattere 6: n
Carattere 7: o
└──n:11
├──u:78
│ ├──-
│ └──o:80
│ ├──-
│ └──-
└──c:15
├──i:35
│ ├──l:50
│ │ ├──-
│ │ └──-
│ └──-
└──a:31
├──-
└──-